Vidyalankar Institute of Technology

Department of Computer Engineering

DEVOPS MINI PROJECT

Week 10 Report

Automated Grocery Inventory Dashboard

Name:	Prachi Patil
Roll No.:	23102B0038
Class:	BE - VII
Division:	B
Academic Year:	2026–27

1. Objective

The objective of Week 10 was to integrate the Selenium test suite of the Automated Grocery Inventory Dashboard into Jenkins and establish a continuous testing pipeline.

The main objectives were:

- Execute automated Selenium tests through Jenkins.
- Publish test results using Jenkins JUnit reporting.
- Ensure that test failures stop package and deployment stages.
- Introduce a deliberate defect and verify that Jenkins detects it.
- Correct the defect and rerun the pipeline successfully.
- Deploy the verified WAR file to Apache Tomcat.

2. Tools and Technologies

Technology	Purpose
Java 21	Application and test execution
Spring Boot 4.1.0	Web application framework
Maven	Build and test automation
Selenium WebDriver	Browser automation testing
JUnit 5	Test framework
Jenkins	Continuous Integration and Testing
MySQL	Application database
Apache Tomcat 10	Application deployment
Git/GitHub	Source code management
Google Chrome	Selenium test browser

3. Continuous Testing Pipeline

A dedicated Jenkins Pipeline job named `Automated-Grocery-Inventory-Continuous-Testing` was configured.

The pipeline executes the following stages:

1. **Checkout** – obtains the latest code from the GitHub branch.
2. **Test** – executes the complete Maven test suite.
3. **Package** – creates the deployable WAR file if tests pass.
4. **Deploy** – copies the WAR file to Apache Tomcat.
5. **Post Actions** – publishes JUnit test results.

The pipeline was configured with an `APP_ENV` parameter containing `development` and `production` options.

The deployment environment used for the successful run was `development`.

3.1 Jenkins Pipeline Configuration

The important part of the Jenkins pipeline is the testing stage:

```
stage('Test') {
    steps {
        sh '''
            export JAVA_HOME=/opt/homebrew/opt/openjdk@21/libexec/openjdk
            export PATH="$JAVA_HOME/bin:$PATH"
            java -version
            chmod +x mvnw
            ./mvnw clean test
        '''
    }
}
```

The package stage executes only after the test stage succeeds:

```
stage('Package') {
    steps {
        sh '''
            export JAVA_HOME=/opt/homebrew/opt/openjdk@21/libexec/openjdk
            export PATH="$JAVA_HOME/bin:$PATH"
            ./mvnw package -DskipTests
        '''
    }
}
```

JUnit reports are published using:

```
post {
    always {
        junit(
            testResults: 'target/surefire-reports/*.xml',
            allowEmptyResults: true
        )
    }
}
```

Thus, if the Test stage fails, the Package and Deploy stages are skipped.

4. Selenium Test Suite

Five Selenium user journeys were integrated into the Jenkins test suite:

ID	Test Case	Verification
TC01	View Inventory	Inventory page and table are displayed
TC02	Add Item	New item is added and displayed
TC03	Search Item	Search result is displayed
TC04	Stock Transaction	Stock-in/out returns quantity to original value
TC05	Exception Alerts	Exception alert functionality is displayed

In addition to these five Selenium tests, the Spring Boot application test was also executed by Maven, resulting in six total tests in the Jenkins run.

5. Deliberate Defect and Failure Verification

To verify that the continuous testing pipeline correctly detects defects, a deliberate error was introduced into the Selenium test.

The expected page title in the inventory test was intentionally changed to an incorrect value. The modified test was committed and pushed to the feature branch.

Jenkins then executed the pipeline. The Test stage detected the failure, causing the later Package and Deploy stages to be skipped.

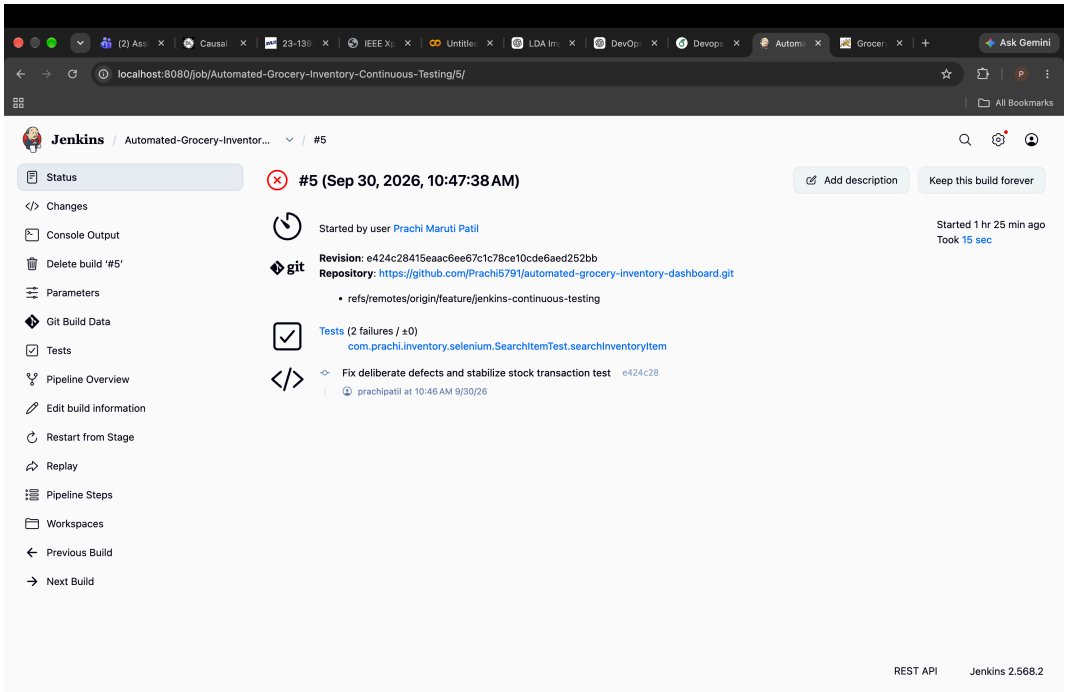


Figure 1: Jenkins pipeline failure after deliberate defect

This demonstrated the required fail-fast behaviour: an unsuccessful automated test prevents the application from being packaged and deployed.

6. Defect Correction

After verifying the failure, the incorrect Selenium assertion was restored to the correct inventory page title.

The stock transaction verification was also corrected so that the test waits for the refreshed inventory row and verifies the final quantity after the stock-in and stock-out operations.

The corrected code was committed with:

```
c10f9bb Fix Selenium stock transaction verification
```

The corrected code was pushed to the branch:

```
feature/jenkins-continuous-testing
```

The local Maven test suite was then executed successfully before the Jenkins rerun.

7. Successful Jenkins Rerun

After the corrections were pushed, Jenkins checked out commit `c10f9bb` and executed the complete pipeline.

The Test stage produced:

```
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

All five Selenium tests passed and the Spring Boot application test also passed.

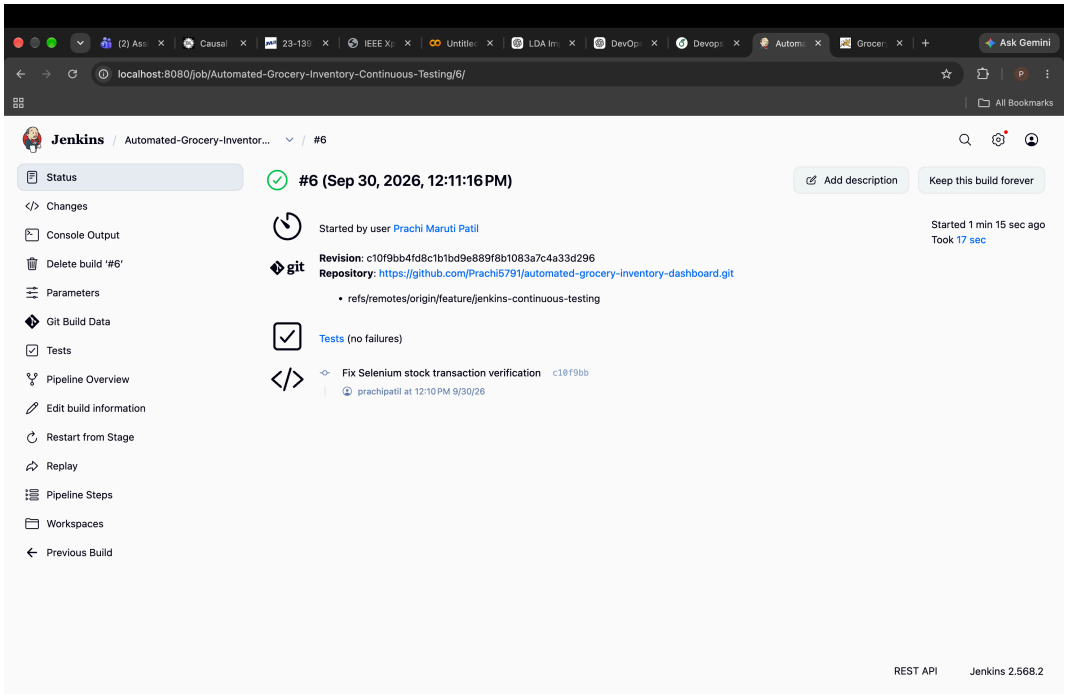


Figure 2: Successful Jenkins pipeline after defect correction

8. Jenkins Test Results

The Jenkins JUnit publisher recorded the Maven test results after the pipeline execution.

The final test result was:

Parameter	Result
Tests Run	6
Failures	0
Errors	0
Skipped	0
Build Status	SUCCESS

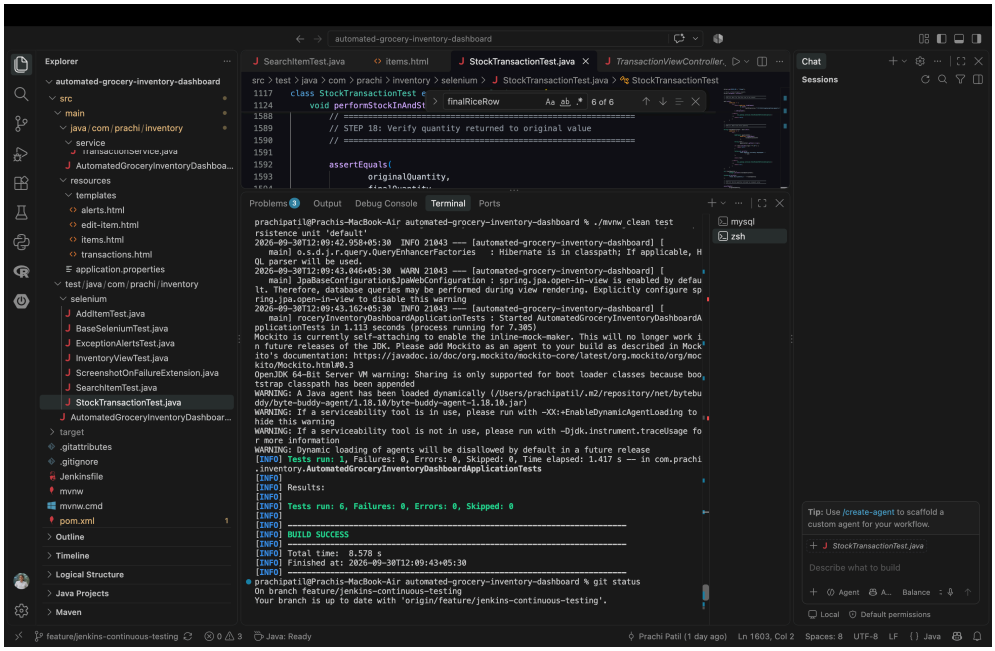


Figure 3: Jenkins JUnit test results

9. Packaging and Deployment

After all tests passed, Jenkins continued to the Package stage.

Maven generated the deployable WAR file:

automated−grocery−inventory−dashboard −0.0.1 −SNAPSHOT . war

The generated WAR file was approximately 51 MB.

The Deploy stage copied the WAR file to the Tomcat webapps directory:

/opt/homebrew/opt/tomcat@10/libexec/webapps/

The application was deployed using the development environment:

APP_ENV = development

Application URL:

```
http://localhost:8082/automated-grocery-inventory-dashboard/items
```

10. Final Pipeline Result

The complete Week 10 pipeline successfully demonstrated continuous testing and controlled deployment.

Stage	**Result**
Checkout	PASS
Test	PASS – 6/6 tests
Package	PASS
Deploy	PASS
JUnit Report	Published
Overall Pipeline	**SUCCESS**

The Jenkins console confirmed:

```
Tests run: 6, Failures: 0, Errors: 0
BUILD SUCCESS
Pipeline completed successfully for environment: development
Finished: SUCCESS
```

11. Conclusion

Week 10 implemented continuous testing for the Automated Grocery Inventory Dashboard using Jenkins and Selenium.

The pipeline was tested with a deliberately introduced defect, and Jenkins successfully detected the failure before packaging and deployment. After correcting the defect, the complete test suite passed with 6 tests and zero failures or errors.

The successful pipeline then packaged the application into a WAR file, deployed it to Apache Tomcat, and published the JUnit test results.

Therefore, the Week 10 requirements for automated testing, failure detection, defect correction, test reporting, and successful deployment were completed.